
#FreeGalane

First Defense

Word Search Project

Nelson Rudent

Matteo Dupond

Noé Francillard

Arnaud Bellicha

November 3, 2024

Contents

I : Introduction	3
II : Group Presentation	4
1 Nelson Rudent	4
2 Matteo Dupond	4
3 Noé Francillard	5
4 Arnaud Bellicha	6
III : Task Distribution	7
1 Interface	7
2 Filtering	7
3 Extraction	7
4 OCR	7
5 Solver	7
6 Reconstruction	8
IV : Progress Status	9
1 Interface - Nelson Rudent	9
2 Filtering - Matteo Dupond	10
3 Extraction - Noé Francillard	12
4 OCR - Arnaud Bellicha	14
5 Solver - Nelson Rudent	16

I : Introduction

This program, created as a project for this S3 (third semester) of EPITA, focuses on developing an application capable of solving a word search puzzle from an input image.

Word search puzzles are common playful problems that require one's brain to search, for each word of a list, the replica inside a grid full of characters. They are not known for putting the players in difficulty. However, the challenge of a program completely solving it from an image involves a much deeper understanding of the problem and difficulty of the task. The application is supposed to replicate each subconscious step that a human makes such as reading the letters, grouping the words and much more. Hence, programming such a project represents a much more interesting topic.

To achieve such goals and expectations, our team “#FreeGalane” is required to be fit for the challenge. Motivation and discipline are both factors that influence the outcome of our adventure. However, our individual strengths and weaknesses constitute the key structure of planning such a project. Each member of the group will present themselves according to these criteria and develop their needs and wants that pushes them to work together and accomplish our ambitions.

II : Group Presentation

It is important that each each member of the group present themselves. They will show their motivations and desires in informatics as well as how it coordinates with their task on this project. Additionally, each member will talk about what the project convey them both professionally and individually along with their explained choice of working with each other.

1 Nelson Rudent

As a student in IT, the path that led me here was my love for programming and the playfulness I felt when solving logical problems with it. Furthermore, I aspired to develop video games to let my imagination and creativity roam free in a world where everything is possible.

This project aligns well with my motivations, as it involves both problem solving within technical limitations. The use of restricted C libraries invoke a closed environment to see how far I can take these limitations to create what I desire.

From a professional view, this project helps me build both essential programming skills in graphical interface and problem solving. On a personal level, it allows me to experiment with innovative approaches, testing my adaptability and discipline when working within constraints.

Working in a group helps me grow as a person and is a rewarding experience because we all bring our strengths and a shared joyfulness to work in cooperation. I chose to work with them because we respect and understand each other's abilities and are able to plan ahead on this behalf.

2 Matteo Dupond

My name is Matteo Dupond, and I have been passionate about programming since I was 15 years old. At that time, I discovered for the first time the ability to create Discord bots using Python. This experience led my interest further down the path of programming. I soon was intrigued in the web development, which allowed me to improve my skills and creativity in building interactive applications. As I continued to grow, I expanded my skills set to other programming languages, such as Node.js and Arduino. This diverse background

provides me with a solid foundation in both backend and frontend development.

Joining this project is an excellent opportunity for me to apply my skills in a practical and professional setting. I am particularly excited about the challenges that lie in the word search puzzle process, where I can contribute to both the technical and creative aspects. My teammates are helpful collaborators that will not only help me grow as a programmer but also encourage a sense of camaraderie as we work towards a common goal. I look forward to improving my experience and interest for programming as we achieve this project together.

3 Noé Francillard

My passion for programming goes back a long way. I first discovered programming when I was 12 years old, at school, with the Scratch code editor. I absolutely loved the fact that you can do anything you want with programming, and the interest I gave to this discovery rocketed over the years. In middle school, I started to attend coding training courses, which included Scratch, Unity or even Python, and I created my first ever programming project, which is a Super Mario Bros. remake on Scratch.

Growing up, my passion for programming pushed me to choose NSI in high school as a speciality, and it was a complete revelation ! I initially wanted to work in video game development, but the opening to the IT universe completely changed my standards. Now, I knew that it was also possible to work on website designing or on artificial intelligence creation ! That is the reason why I went to EPITA : since there were many professional opportunities in the information technology (IT) domain, I wanted to study in a school that permits its students to fully explore the IT universe ! Therefore, thanks to that choice, I discovered website designing, ethical hacking, quantum technology and image treatment.

Thanks to the S3 project, I was able to discover this last domain, image processing. This project will allow me to compare my expectations with reality and see if I want to work in the image treatment domain after my studies. Furthermore, this project is the most difficult task I was ever assigned until now, which implies that problem resolution is going to be more difficult, and hence more challenging. This is a great opportunity to challenge myself, push my boundaries further than ever and unlock new skills, such as a stronger determination, better common sense and logic or, more simply, new programming skills.

Moreover, since this project pushes us to find new ways to think, it will be the key to learn new thinking schemes and principles that will be necessary to resolve more difficult problems.

The most interesting part of the project is the fact that it is not individual. Here, we can learn how companies manage dozens of projects by doing a project ourselves ! This is also the chance to build new collective soft skills. Indeed, doing a project with three other students is the bedrock to learn how to listen to each other, be patient and develop team spirit. And people learn even more when their colleagues are also their friends. That is why we decided to form this team : because each member of the group works, but also brings joyfulness to the other project members and motivates them. Being friends in a project group is a non-negligible advantage, as we already know each other, and are going faster and further than other groups in which team members do not know each other.

4 Arnaud Bellicha

As a 19-year-old second-year student at EPITA, my name is Arnaud Bellicha. The place where I feel most at ease is behind a computer, which is why I decided to pursue a career in this field. I am highly curious, and my goal in joining EPITA was to expand my skills and explore various subfields within computer science, such as artificial intelligence, machine learning, and cybersecurity. I also have a strong appreciation for problem-solving, as I find it satisfying to believe that every problem has a solution, it is even more rewarding to find it.

I see this project as a chance to look more into neural networks and work on the seemingly straightforward problem of a Word Search by carefully constructing and solidifying each step of a development process. Professionally, I am eager to sink myself in the artificial intelligence environment, practicing and understanding the associated mechanics. Personally, I aim to strengthen my teamwork skills, working in cooperation with my teammates. On a human level, our group is formed in our early weeks at EPITA, and this year, we are ready to push our limits to achieve a high-quality project.

All in all, I am very happy about completing this project with my team and gaining new skills and knowledge with it.

III : Task Distribution

Our project involves a multi-step process to load, analyze, and solve a word search puzzle based on an input image. Hence, to coordinate our group, the task distribution is based on each their own skills and interests and is divided into the following categories:

1 Interface

Involves, in the foreground, visually appealing and interacting graphics, as well as, in the background, connecting the subsystems of the application together to produce the final product. The task is assigned to the member Nelson Rudent as it embraces well their strengths of working in a closed environment and it also fits their group leader role for the managing part.

2 Filtering

Contains the processing of the input image through various filters and algorithms. Since this part requires a strong skill set on image processing, Matteo Dupond has been selected as they posses experience on similar tasks, notably in their web development journey.

3 Extraction

This step happens to be the most challenging discrete obstacle of the solving process. It requires high motivation and interest to push their own limits and find a solution to detect with an alogrithm only the correct positions. A person capable of this would be Noé Francillard who loves to be confronted with difficult challenges.

4 OCR

The optical character recognition part of the project consists of developing an artificial intelligence capable of reading with high accuracy a letter. For this reason, Arnaud Bellicha provides a strong foundation and interests in the subject.

5 Solver

The solver is an independent executable for which the goal is to search a word inside a grid character. It must be well coordinated

with the Interface. Nelson Rudent fits this role due to his problem solving capacities and him being in charge of the Interface too.

6 Reconstruction

The reconstruction task which involves drawing on an image to highlight or cross a word, is similar in requirements than of the Filtering task. For this reason, Matteo Dupond is the perfect candidate due to their image processing skills.

IV : Progress Status

For the first defense, the expected progress across the tasks varied. The Interface development advanced far beyond expectations, as we recognized its critical role in connecting all application subsystems. Originally, we aimed to create a basic interface to load an image and call the filtering and extraction subsystems, but it has already achieved much more integration. Filtering on the other side faced minor delays; we intended to have more filters ready but encountered challenges in fine-tuning constants for different cases.

Our worse take was in the extraction which is significantly delayed because we underestimated its complexity and got distracted by searching the implementation of the artificial intelligence. As a result, extraction currently works only for grids with lines and has yet to handle cases without lines or extract the word list. OCR is progressing as expected, nearly achieving the goal of learning the expected logical equation. Finally, Solver surpassed initial expectations, not only completed but also fully integrated into the application.

1 Interface - Nelson Rudent

Before creating the interface and even before planning its final product, the tools to use were a mandatory choice. After a test session and comparison analysis between SDL and GTK3, we decided to use GTK3. Thanks to its widget structure, it is a much more advantageous library for developing quickly an application interface. Furthermore, it also supply our group with countless implemented tools such as GDK, Cairo, Pango and a lot more. To use the library, it must be initialize with `gtk_init()` and then run the main application loop with `gtk_main()`.

However, to hold the interface, a window is required. For most of the library, GTK3 follows the same architecture: "`gtk_xxx()`". Hence, to create a new instance of a `GtkWindow`, we use `gtk_window_new()`. Likewise, the title is set with `gtk_window_set_title()`, the default size with `gtk_window_set_default_size()` and to maximize it we use `gtk_window_maximize()`.

To create the window the size of the screen, playing with the functions `gdk_monitor_get_geometry()` was the solution. Using a `GdkMonitor` instance, the screen details could be retrieved.

GDK is an integrated tool of GTK which overall handles the win-

dow system. Thus, we call `gdk_display_get_default()` as well as `gdk_display_get_monitor()` to seize the required dimensions.

However, we are not yet able to construct our interface based on those dimensions as they cover the size of the monitor in pixels but do not take into consideration the window size nor the window decorations, also known as the contour.

To operate our application, GTK is structured with signals or events. Those are sometimes dependent on one another and of the type of elements they are associated with. They can be assigned to any functions with the `g_signal_connect()` function family.

Using this trick, we intercept the "size-allocate" signal and grab the content area's dimensions using `gtk_window_get_size()`. Since the operating system of the computer manipulates the window according to their rules, we must avoid fake "size-allocate" signals by checking if the retrieved dimensions are inferior to the window size. To prevent building the application twice, we disconnect the signal with `g_signal_handlers_disconnect_matched()`.

The first step is taken care of by the interface itself. To provide an enhanced experience for the user, a file selector dialog would be the right choice. This is possible thanks to the provided GTK function `gtk_file_chooser_dialog_new()`. It creates an instance which, using the right parameters, sets it to import an image file. When the dialog is run using `gtk_dialog_run()`, it puts the program in waiting mode and only allow the user to continue by replying to the dialog. When this happen, the response is sent back from the function.

If it is a positive response, we grab the filename of the image the user wants to load. To perform this task, a specific type is necessary in GTK. Indeed, the `GdkPixbuf` represents a numerical image that can be processing in almost every way. GTK provides predefined functions for standard operations on this type, such as `gdk_pixbuf_new_from_file()`. Furthermore, we can pair this technique with the `GtkImage` widget, which can display our `pixbuf` on the application. An instance of this widget is then created by calling `gtk_image_new_from_pixbuf()`.

2 Filtering - Matteo Dupond

In the filtering phase of our project, we focus on converting the input image to grayscale and then applying a binary threshold to pre-

pare it for further processing. This initial step is crucial for enhancing the clarity of both the grid and the word list, enabling more accurate extraction and recognition in the following steps.

The current implementation integrates a grayscale conversion directly available inside the application. The first step is to convert the input image to grayscale. This simplifies the color data, reducing it to a single channel, which is essential for the next processing stages. Hence, the written function `grayscale_pixbuf()` handles this conversion using a weighted sum of the RGB components. It is provided by the following formula:

$$gray = 0.3 \times R + 0.59 \times G + 0.11 \times B$$

This equation accounts for human perception, where green contributes the most to the perceived brightness. After converting to grayscale, we apply a binary threshold to create a black-and-white image. Pixels above the specified threshold are set to white (255), and those below are set to black (0). This step is critical as it highlights the letters against the background, not only simplifying the detection but required by it.

To achieve both image processing algorithm, the implementation iterates through each pixel. It updates their values based on the thresholding criteria. This is done within nested loops that traverse the image dimensions, accessing pixel data via a pixel buffer, which itself is given from the `GdkPixbuf` type.

While our current approach establishes a solid foundation for image filtering, there are several parts we could improve and make more efficient.

Indeed, one significant improvement involves implementing Sauvola thresholding. Unlike simple global thresholding, Sauvola adapts the threshold based on local pixel intensity variations, which can be particularly advantageous for images with uneven lighting or complex backgrounds. This method calculates a local mean and standard deviation, allowing for a more nuanced distinction between foreground (letters) and background.

As illustration, pink colored letters would still be kept in the image which is not the case for the grayscale standard thresholding. Its formula contains the threshold at a given pixel, the local mean, the local standard deviation, and a sensitivity factor. By using this method, we can improve the accuracy of the letter extraction process, particularly

in challenging lighting conditions.

Another key element to improve would be the implementation of noise reduction techniques prior to thresholding. Images often contain noise: random variations in brightness that can obscure the true pixel values. By applying techniques such as Gaussian blurring or median filtering, we can smooth out these irregularities. Gaussian blurring uses a twist with a Gaussian kernel to reduce high-frequency noise, while median filtering replaces each pixel value with the median of the values in its neighborhood. This preprocessing step can significantly make a difference in the image quality.

We could also consider incorporating morphological operations such as dilation and erosion. These operations can help refine the shapes of the letters, making them more distinct by removing small noise points (erosion) or connecting disjointed components (dilation).

3 Extraction - Noé Francillard

In the extraction step, as previously explained in the interface, we cannot directly use the image and have to work with a `GdkPixbuf`. It essentially is an array that contains all the pixels data. We can create a `pixbuf` from a file with `gdk_pixbuf_new_from_file()`.

Three structs were created in order to simplify the general handling of this step. First, the structure `Position` has two integer attributes, `x` and `y`, representing coordinates. The other structure, `Line`, has two `Position` attributes rendering the initial and end position of a line. The final structure, `BoundingBox` also contains two `Position` attributes marking the top left and bottom right coordinates of a box.

The goal is then to detect and save the position of the grid, the word list and all the letters in both of them. With custom functions, we detect and return each line of the letter grid (if existing) as a 'Line' structure in a heap-allocated 2D-array. If needed, the `search_for_grid()` function can create the top, down, left and/or right side of the grid, as it will be difficult to gather information about letter positions if the grid has no outlines.

Since we do not know how many lines there will be, there may be some memory reallocation needed. Additionally, given that the length of the horizontal and vertical lines will be used in outer functions, we need to store them directly into the addresses of two variables where pointers can trace them back.

The other technical methods are used in the `horizontal_search()` and `vertical_search()` functions. They both return all the lines of the grid in the correction direction, and stores the number of them.

Because we are working with the `pixbuf` structure, we can grab its details such as the dimensions using the standard GTK functions `gdk_pixbuf_get_width()`, `gdk_pixbuf_get_height()` as well as `gdk_pixbuf_get_pixels()`.

Since we don't yet know the number of horizontal or vertical lines, we define a heap-allocated array to store them, using the `malloc()` function, allowing resizing.

In order to search for a pixel, you have to identify the number of bytes between the start of a row and the start of the next row : the rowstride, given by `gdk_pixbuf_get_rowstride()`. To iterate pixel per pixel, we have to go `N` elements further in the `pixbuf` array, taking into account the number of elements per pixel. This attributes `N` can be gathered with `gdk_pixbuf_get_n_channels()`.

Thus, we can access each pixel with the following formula :

$$p = \text{pixels} + \text{current_y} * \text{rowstride} + \text{current_x} * \text{n_channels};$$

If no horizontal lines are found, the allocated memory is freed before returning `NULL`. Otherwise, it is resized to be large enough to contain the new values.

Further functionalities have been coded to create grid outlines if needed on the top, down, left and/or right side of the grid. We can also detect if there is a missing outline on one side of the grid, compare the `x` (resp. `y`) coordinate of the row and the `y` (resp. `x`) coordinate of the column from the same corner and add an outline.

That said, when the pixel positions are not perfectly accurate, the coordinates comparison will be distorted, and results will be false. We can include an error margin of 5 pixels so that, even if the coordinates are not perfect, the program will still be able to determine if an outline is needed to be created on one side of the grid.

The use of the `memcpy()` function allows us to store the updated data into the original, reallocated array. Similarly, to separate and save a resulting letter, line or word, we create a `subpixbuf` from the original one, essentially cutting it. This process can be realised with `gdk_pixbuf_new_subpixbuf()`. Since their data is shared, we have to

create and return a copy of the cutted pixbuf.

When it comes to saving the collected data, we target a determined folder with a determined name. The format of the newly created image is "png". Only the size of the array, symbolizing the path, changes. The `strcpy()` function is very useful to store a non-empty string into another bigger, empty string without making any complicated loops. Similarly, the `strcat()` function is very useful to store a non-empty string into another bigger, non-empty string.

The only remaining operation to do at the end of the function is to save the pixbuf which can thankfully be automatically done using the GDK function `gdk_pixbuf_save()`.

4 OCR - Arnaud Bellicha

A neural network is a complex structure made of perceptrons. A perceptron is a node that takes several inputs, which are either 0 or 1, and produces an output value that is also either 0 or 1.

$$\text{output} = \begin{cases} 0 & \text{if } w \cdot x + b \leq 0, \\ 1 & \text{if } w \cdot x + b > 0. \end{cases}$$

Each input value has a weight that corresponds to its importance, along with a bias, which is an arbitrary parameter fixed for the perceptron. The value of the output is determined by this method:

Where

$$w \cdot b = \sum jw_jx_j$$

is the weighted sum of inputs.

This is the first version of a neuron that I discovered while researching. I later learned that for our network to learn and adapt by changing its weights and biases, we needed another type of node. This is the primary aim of a neural network: to learn to solve problems by adapting itself through adjustments in weights and biases. This algorithm is called gradient descent, which helps fix errors in the network after each prediction in order to achieve correct results over time. To accomplish this, we want to induce a small change in output through small adjustments in weights and biases. However, the perceptron could result in significant changes in output.

This is when I discovered the sigmoid neuron. It follows the same concept as a perceptron but takes inputs between 0 and 1 and returns the following output $\sigma(w \cdot x + b)$, denoted by the activation function of a neuron.

$$\sigma(z) \equiv \frac{1}{1 + e^{-z}}.$$

This is the sigmoid function that will be used in our neural network. This type of node allows us to produce a small change in output according to small changes in biases and weights. Now that we have the correct node, we can construct our neural network, which is divided into three parts: the input layer that contains our inputs, the output layer that provides our results, and the hidden layers in between.

Our network will operate based on two main concepts. On one hand, there is the Forward Propagation: the network processes inputs layer by layer, applying weights, biases, and activation functions to generate outputs. On the other hand, there are the Backpropagation and the Weight Updates: the network calculates errors, updates weights, and adjusts biases layer by layer, enabling it to learn over time.

To implement this structure, we will create sub-structures. We have a Neuron structure that holds an array of weights representing the connections between neurons from previous layers and the current neuron. The neuron also has its corresponding bias, and slots to store future output and errors computed by the Backpropagation.

The layer structure will also be useful. It contains an array of neurons and the number of neurons within that layer.

Finally, the NeuralNetwork structure, which is our main structure, contains an array of layers and the number of layers.

We can now interact with our NeuralNetwork structure, creating functions around it. We initialize the neurons of each layer by assigning random biases and weights, a questionable choice that we may reconsider for the final OCR.

The functions for Backpropagation, Forward Propagation, and Weight Update can now be implemented. The Forward Propagation function for each node in each layer calculates the weighted sum and applies the activation function to store the output of the neuron. The Backward Propagation function computes error deltas between the obtained values and the expected outputs, propagating these errors to the hidden layer.

Output Layer Delta:

$$\delta = (\text{expected} - \text{output}) \times \sigma'(\text{weighted_sum})$$

Hidden Layer Delta:

$$\delta = \left(\sum \delta_{\text{next layer}} \times \text{weight}_{\text{to this neuron}} \right) \times \sigma'(\text{weighted_sum})$$

The deltas are stored in the neurons. To update our weights, we can use the computed deltas in the following manner to derive new weights: $\text{new weight} = \text{weight} + (\text{learning rate} \times \text{delta} \times \text{input})$. The learning rate is a fixed parameter for our network, and setting it to 0.1 seems to work well for the XOR problem; we will see how it performs for character recognition.

Finally, we will repeatedly call these three functions within a training function for a certain number of iterations, referred to as EPOCHS, which is another fixed parameter.

To verify the proper functioning of my network, I also implemented test functions that print the values obtained by the neural network. One way of improve our network would be to generalize the sizes of our inputs and use a Softmax instead of the classical activation function in the output layer.

5 Solver - Nelson Rudent

For proper use, the first step of the Solver is to check if the correct number of arguments is given through the use of “argc”. Then, before attempting to process any dynamic research, the given file is entirely read and a grid is constructed from it through `read_file()`.

This function determines the size of the grid first which is necessary for its dynamic allocation. Indeed, after retrieving the correct size, the grid, an array of arrays of characters, is dynamically allocated using `malloc()` as seen in practicals. Since we had to read the file to extract the necessary grid dimensions, the reading cursor has been moved. Calling `fseek()` reset the cursor at the start of the file.

From this point on, the file is read using the standard function `getline()` which stores in a buffer a line of the file. A line is considered to be the largest string that you can read of the file before finding a newline character or the null terminator.

For each line, the cell of the buffer containing the newline character is replaced by the null terminator. This allows us to use `toupper_str()`, a personal function which calls `toupper()` on every letter of the array.

After this process, the buffer is copied into the correct grid line using `strcpy()`.

Now that we possess the grid, it was time to search the word in it: `search_word()`. This function iterates over the entire grid. When it finds the first corresponding character, it calls `search_direction()` in every possible direction and stops the loop if successful. The latter function checks in a direction for each correct letter of the word, in order. On success, it updates the pointed struct given as arguments.

The struct “coords” defined to handle the positions simply contains the row and column of a letter. Hence, the Solver finds the starting position of the word and its ending position and prints the result on the standard output stream (`stdout`). If nothing is found, it prints the default error “Not found”.